evaluator.py

Validation metrics: AUC, LogLoss, PR-AUC, calibration, and lift

---

All evaluation in this project routes through `evaluate()`, called once per trained pipeline against the held-out validation split. Two additional diagnostic functions (`calibration_table`, `lift_at_k`) are available but not currently wired into the main training scripts — they're for deeper, ad hoc inspection of a model's predicted probabilities.

evaluate FUNCTION

```
evaluate(y_true, y_pred_proba) -> dict
```

Computes three complementary metrics via scikit-learn, all from the true binary labels and the model's predicted probability of the positive class (`click = 1`):

- **AUC** (`roc_auc_score`) — ranking quality, independent of any classification threshold or probability calibration.
- **LogLoss** (`log_loss`) — a proper scoring rule that penalizes both wrong *and* overconfident predictions.
- **PR-AUC** (`average_precision_score`) — ranking quality focused on the positive (minority) class, more informative than AUC under severe class imbalance (Avazu's overall CTR is ~16–17%).

AUC — the probability a randomly chosen positive example is ranked above a randomly chosen negative one

$$\text{AUC} = P(\hat{p}_{i^+} > \hat{p}_{j^-}) = \frac{\sum_{i \in \text{pos}} \text{rank}(\hat{p}_i) - \frac{n_+(n_+ + 1)}{2}}{n_+ n_-}$$

LogLoss (binary cross-entropy) over N validation rows

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

PR-AUC (average precision) — a step-weighted area under the precision/recall curve

$$\text{PR}-\text{AUC} = \sum_n (R_n - R_{n-1}) P_n \,, \qquad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

`print_metrics` FUNCTION

```
print_metrics(metrics: dict) -> None
```

Formats each entry of the metrics dict as `name: value` with four decimal places — the plain-text summary printed after every `trainer.py` run.

`calibration_table` FUNCTION

```
calibration_table(y_true, y_pred_proba, n_bins=10) -> pd.DataFrame
```

Buckets rows into `n_bins` equal-*population* quantile bins by predicted probability (`pandas.qcut`, i.e. each bin has roughly the same row count rather than the same probability width), then reports, per bin, the mean predicted probability, the mean actual label, and the bin size.

This is the standard reliability-diagram construction: a well-*calibrated* model has mean-predicted ≈ mean-actual within every bin, even if its AUC/ranking quality is identical to a poorly-calibrated model with the same ranking of scores.

`lift_at_k` FUNCTION

```
lift_at_k(y_true, y_pred_proba, k_fractions=[0.05, 0.1, 0.2]) -> dict
```

Sorts all rows by predicted probability descending, then for each fraction `k` in `k_fractions` takes the top `⌊N·k⌋` rows and computes their empirical CTR relative to the overall (baseline) CTR:

A lift of, say, 3.0 at `k=0.05` means: if you could only show ads to the top-ranked 5% of impressions by predicted score, you'd see 3× the click-through rate of showing ads unconditionally — a business-facing way to read a ranking model's usefulness at a specific operating point, independent of any single classification threshold.

Lift at k

$$\text{lift}(k) = \frac{\text{CTR}_{\text{top}-k}}{\text{CTR}_{\text{overall}}}$$